## 1 Five-Person Plan: Roles, Duties, Research Tasks, and Tech Stack

The work splits cleanly across five people with minimal overlap. One person (Research/Finance) owns the research-grade problems and ships reference-grade libraries/specs that the engineers integrate. One person is dedicated to the front end. The remaining three roles cover (i) smart contracts/on-chain verification, (ii) backend delivery services (scheduler, QC oracle, settlement driver), and (iii) provider runtime (GPU execution + overlay + infra). The goal is to complete an end-to-end demo on four RTX 3060s while keeping the H100 NGH baseline in the spec.

**Role 1: Research & Finance (Theory + Minimal Reference Implementations).** This person defines the quantitative and cryptographic "source of truth" for the platform and supplies minimal, correct reference implementations that the backend, contracts, and UI can safely rely on. They are explicitly not the primary implementer of the provider runtime or the contract code, but they *do* own the canonical encodings, verification rules, market methodology, and research deliverables that prevent the engineering system from drifting. *Primary research tasks and deliverables (mapped 1:1 to v2):*

- **QC canonicalization & tolerant equivalence (§2.1, §2.4.5–§2.4.6).** Ship `packages/canonicalization/c` (Python): deterministic record ordering, streaming output hashing, ULP-aware float comparator (NaN/Inf safe), plus a gold corpus and adversarial deltas. *Done when:* false rejects and false accepts are below targets on the corpus; comparator is fast enough to run in QC paths; integrated behind a flag.
- **Verification sampling & dispute math (§2.1, §2.4.5–§2.4.6, P5).** Ship `apps/api/app/services/qc/` simulator + analytic bounds that translate $(\epsilon, \alpha, \beta)$ into (dup/canary/spot) and dispute re-exec fraction $q$; output a policy JSON consumed by the QC oracle. *Done when:* empirical detection power meets the bound under adversarial strategies at the chosen overhead.
- **CADO-NFS determinism & certificate proof-kit (§4, P5).** Ship `cado-proofkit/`: bit-exact settings where possible, test vectors (small $N$ with known factors; tiny matrices with known kernels), stage-by-stage expected hashes, and a certificate verifier for $Mv = 0$ over $\mathbb{F}_2$ plus final gcd check. *Done when:* independent runs yield identical artifacts under bit-exact mode and the verifier catches synthetic corruptions with 100% recall.
- **Gang-aware placement feasibility oracle (§2.3.2, §2.7, P3).** Ship `collective-feas/`: given provider capabilities and measured RTT/bandwidth samples plus a session descriptor, output feasible/infeasible and a rank→provider suggestion with predicted start-skew/RTT/bandwidth quantiles. *Done when:* predictions correlate with observed pass/fail in replay runs and false-feasible rate is low.
- **Index builder & short-tenor futures pricer (§2.2.3, §2.4.7, P6, D9).** Ship `market-index/` (hourly VWAP of accepted NGH per key tuple with depth floors/outlier rules) and `fut-pricer/` (simple nowcast curve with seasonality + basis). *Done when:* low-volume stability is documented and manipulation simulations cannot move the index beyond $\gamma$ without significant cost.
- **Normalization & anti-gaming (§2.2.2, P4).** Ship `qbench/`: schema-specific microbenchmarks and a raw GPU-time estimator excluding idle/I/O; drift detection when benchmark score diverges from realized throughput. *Done when:* bench throughput correlates strongly with realized throughput and basic gaming attempts are detected.

*Additional cross-cutting "interface" work (critical for on-chain verification and multi-provider Collective):*

- **H100 NGH baseline calibration protocol (DemoNet on 3060s, Production NGH on H100).** Publish the H100 baseline score table (by schema/benchmark), the exact benchmark conditions, and the reproducible pipeline for computing $q$ on 3060s while keeping H100 as the NGH definition. Provide confidence bounds and documented assumptions.
- **Canonical leaf encoding spec for `receipt_root` and `qc_root`.** One-page strict encoding: field order/types/endian-ness, domain separation, float avoidance (scaled ints), map sorting, and Merkle construction rules. Ship a Python reference encoder/decoder.
- **Proof bundle format + verifier CLI (`proofkit/`).** Emit/verify inclusion proofs for receipts and QC events against on-chain roots; support batch verification for demos and UI integration.
- **Settlement invariants + trace vectors.** Specify exactly-once consumption, correct key tuple consumption, idempotent anchor/settle, and replay determinism; produce JSON "ledger traces" usable as contract property tests.
- **Collective security + telemetry truth model.** Define membership proofs, session keying, ACL rules, telemetry commitment (`net_telemetry_root`), and objective degradation/slashing triggers; ship reference hashing/signing code for telemetry bundles.
- **Threat model + attack suite.** Enumerate practical attacks (wrong output, selective cheating, milestone gaming, voucher mismatch, overlay ACL bypass, bogus telemetry) and map each to detection evidence and settlement/slash outcomes.

*Tech stack:* Python (numpy/pandas/scipy), small cryptographic/Merkle utilities (hashlib + custom library), Jupyter for experiments, LaTeX for spec, and a shared schema/ABI repo. Output artifacts as versioned JSON/CSV plus reference libraries.

**Role 2: Front-End Engineer (Buyer/Provider UX + Demo Dashboards).** This person builds the web product that drives the entire demo and makes the system observable. Duties:

- Buyer flow: select (region, hour, SLA, tier), acquire exposure (demo), view vouchers, create jobs/sessions, deposit vouchers, track gate close and prepare readiness, view QC results, download and verify receipt bundles against on-chain roots.
- Provider portal: provider capability view, nominations submission, assigned lots list, status progression, log links.
- Operator views: QC events, disputes, settlement events, hourly indices and near-tenor curve snapshots.

*Tech stack:* Next.js (React + TypeScript), UI library (shadcn/ui or MUI), React Query, SSE/WebSockets, recharts, and (if needed) wallet tooling (wagmi + viem).

**Role 3: Smart Contracts Engineer (Voucher + Anchoring + Settlement).** This person implements the on-chain state that makes vouchers and verification auditable and settlement enforceable. Duties:

- Voucher token keyed by (region, hour, SLA, tier) with minting hooks (demo clearing), transfers,

and job escrow/deposit.

- ReceiptRegistry anchoring `receipt_root` and `qc_root` with events for indexing and audit.
- Settlement: consume deposited vouchers 1:1 with accepted paid_NGH; pay providers (demo ledger acceptable); slash on QC failures; enforce idempotency and exactly-once semantics.
- Invariant tests (Foundry): no double-consume, consume-only-if-accepted, correct-key tuple only, replay/idempotent traces.

*Tech stack:* Solidity, Foundry (preferred) or Hardhat, Anvil for local chain, event indexing via backend listener; ABI pinning.

**Role 4: Backend Platform Engineer (Scheduler + Job Service + QC Oracle + Settlement Driver).** This person builds the delivery engine that turns nominations and vouchers into verified compute. Duties:

- APIs: jobs, sessions, feasibility, voucher deposit status, receipts bundle download, and realtime status feed.
- Scheduler: ingest nominations; assign packages at gate close; track prepare readiness; wave control; reassignment on prepare/first-output misses; (Collective) enforce session membership fixation and feasibility constraints from the oracle.
- QC oracle: duplication/canary/spot selection, dispatch neutral pool tasks, dispute ladder execution, accept/reject decisions, QC events.
- Anchoring/settlement driver: compute roots, post to contracts, invoke settlement with accepted paid_NGH per provider; publish indices snapshots.

*Tech stack:* FastAPI (Python) or NestJS (TypeScript), Postgres, Redis + queue (Celery/RQ or BullMQ), SSE/WebSockets, and shared libraries from Role 1 (canonical encoders, Merkle proofs, comparators, index builder).

**Role 5: Provider Runtime & Infrastructure Engineer (GPU Execution + Overlay + Neutral Pool).** This person makes compute actually run on GPUs and produces runtime telemetry and overlay connectivity for Collective SLA. Duties:

- Provider agent: pull image digest, prefetch inputs during prepare, enforce basic sandbox/egress policy, run container pinned to one GPU, emit heartbeat/first output, measure raw GPU time and wall time, upload logs, submit results.
- Collective overlay module: session join/leave, members-only ACL + allowed ports, encrypted tunnels, teardown at hour end, telemetry sampling and commitment (root + signatures).
- Neutral pool runner for spot checks and disputes; local deployment tooling (docker-compose or minimal k8s) and observability.
- Multi-provider emulation on a single machine by running four provider identities bound to GPUs 0–3; optional latency injection via `tc netem`.

*Tech stack:* Python or Go, Docker + NVIDIA Container Toolkit, NVML/`nvidia-smi` (pynvml), container sandboxing primitives, `tc netem`, structured logs (optional Prometheus/Grafana).

**Team Definition of Done (Demo-Level).** A complete end-to-end run succeeds on four RTX 3060 GPUs using the DemoNet parameterization while preserving the H100 NGH definition: the buyer acquires a deliverable position, vouchers mint and are deposited, multiple providers nominate and are assigned at gate close, prepare completes within the bounded window, packages start at $T_0$ and emit first output/heartbeat within 2 minutes, QC produces accept/reject decisions with at least one duplicate and one spot-check, `receipt_root` and `qc_root` are anchored on-chain, settlement consumes the correct voucher keys 1:1 with accepted paid_NGH and pays/slashes providers, and the buyer downloads a receipt bundle that verifies against on-chain roots. For the reference CADO-NFS demo, the final factor is verified (gcd check) and archived together with its certificates, canonical hashes, and proofs.